WIRESHARK CHECKSUM VALIDATION - DOCUMENTATION

IPv4:

Packet chosen:

No.	Time	Source	Destination	Protocol	Info	Length	Source Port	Destination Port	Source HW Address	Destination HW Address
1	0.000000	172.20.40.252	182.79.245.183	TCP	49947 → 443 [ACK] Seq=1 Ack=1 Win=2048 Len=0	54	49947	443	82:39:5e:f8:7a:7b	Cisco_61:f5:1f

Wireshark checksum:

```
Internet Protocol Version 4, Src: 172.20.40.252, Dst: 182.79.245.183
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
  > Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 40
    Identification: 0x2874 (10356)
  > 000. .... = Flags: 0x0
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 64
    Protocol: TCP (6)
    Header Checksum: 0xd144 [correct]
    [Header checksum status: Good]
    [Calculated Checksum: 0xd144]
    Source Address: 172.20.40.252
    Destination Address: 182.79.245.183
    [Stream index: 0]
```

Manual calculation:

1. On wireshark right click IPv4, copy->hex stream to get IPv4 header bytes:
   4500002828740000 4006d144ac1428fcb64ff5b7
2. Split into 4 hex digits each:
   4500
   0028
   2874
   0000
   4006
   d144 ← (original checksum)
   ac14
   28fc
   b64f
   f5b7
3. Replace the checksum with 0000
   4500
   0028
   2874

0000
4006
0000
ac14
28fc
b64f
f5b7

4. Add all these values
   4500 + 0028 = 4528
   4528 + 2874 = 6d9c
   6d9c + 0000 = 6d9c
   6d9c + 4006 = ada2
   ada2 + 0000 = ada2
   ada2 + ac14 = 159b6
   This exceeds 16 bits so wrap around carry
   59b6 + 1 = 59b7
   59b7 + 28fc = 82b3
   82b3 + b64f = 13902
   Wrap around
   3902 + 1 = 3903
   3903 + f5b7 = 12eba
   Wrap around
   2eba + 1 = 2ebb
   Final Sum: 2ebb
5. 1’s complement:
   ffff - 2ebb = d144
   Checksum is valid

TDP:

Packet chosen:

No.	Time	Source	Destination	Protocol	Info	Length	Source Port	Destination Port	Source HW Address	Destination HW Address
1	0.000000	172.20.40.252	182.79.245.183	TCP	49947 → 443 [ACK] Seq=1 Ack=1 Win=2048 Len=0	54	49947	443	82:39:5e:f8:7a:7b	Cisco_61:f5:1f

Wireshark checksum:

```
Transmission Control Protocol, Src Port: 49947, Dst Port: 443, Seq: 1, Ack: 1, Len: 0
    Source Port: 49947
    Destination Port: 443
    [Stream index: 0]
    [Stream Packet Number: 1]
  > [Conversation completeness: Incomplete (60)]
    [TCP Segment Len: 0]
    Sequence Number: 1    (relative sequence number)
    Sequence Number (raw): 1774583482
    [Next Sequence Number: 1    (relative sequence number)]
    Acknowledgment Number: 1    (relative ack number)
    Acknowledgment number (raw): 209285273
    0101 .... = Header Length: 20 bytes (5)
  > Flags: 0x010 (ACK)
    Window: 2048
    [Calculated window size: 2048]
    [Window size scaling factor: -1 (unknown)]
  > Checksum: 0x7c53 [correct]
    [Checksum Status: Good]
    Urgent Pointer: 0
  > [Timestamps]
    [Client Contiguous Streams: 1]
    [Server Contiguous Streams: 0]
```

Manual calculation:

1. On wireshark right click TCP, copy->hex stream to get TCP header bytes:
   c31b01bb69c5feba0c797099501008007c530000
2. Break into 16-bit words:
   c31b
   01bb
   69c5
   feba
   0c79
   7099
   5010
   0800
   7c53 ← checksum
   0000
3. Replace checksum with 0000
   c31b
   01bb
   69c5
   feba
   0c79
   7099

5010
0800
0000
0000

4. Construct TCP Pseudo Header

Source IP: 172.20.40.252

Convert to hex:
172 = AC
20 = 14
40 = 28
252 = FC
So,
ac14
28fc

Destination IP: 182.79.245.183

Convert to hex:
182 = B6
79 = 4F
245 = F5
183 = B7
So,
b64f
f5b7

Reserved + Protocol

Reserved = 00
Protocol = 06

So:
0006

TCP Length

TCP header = 20 bytes
Data = 0

So:
0014

Final Pseudo Header:
ac14

28fc

b64f

f5b7

0006

0014

5. Add Pseudo header and TCP header(with checksum 0000)

ac14

+28fc

= d510

d510

+b64f

= 18b5f → wrap

8b5f +1 = 8b60

8b60

+f5b7

= 18117 → wrap

8117 +1 = 8118

8118

+0006

= 811e

811e

+0014

= 8132

8132

+c31b

= 1444d → wrap

444d +1 = 444e

444e

+01bb

= 4609

4609

+69c5

= afce

afce

+feba

= 1ae88 → wrap

ae88 +1 = ae89

ae89

+0c79

= bb02

bb02

+7099

= 12b9b → wrap

2b9b +1 = 2b9c

2b9c

+5010

= 7bac

7bac

+0800

= 83ac

83ac

+0000

= 83ac

83ac

+0000

= 83ac

Final Sum: 83ac

6. 1’s complement

ffff - 83ac = 7c53

Checksum is valid

UDP:

Packet chosen:

udp

No.	Time	Source	Destination	Protocol	Info	Length	Source Port	Destination Port	Source HW Address	Destination HW Address
413	2.288355	172.20.40.252	142.251.150.119	UDP	49939 → 443 Len=29	71	49939	443	82:39:5e:f8:7a:7b	Cisco_61:f5:1f

Wireshark checksum:

```
User Datagram Protocol, Src Port: 49939, Dst Port: 443
    Source Port: 49939
    Destination Port: 443
    Length: 37
    Checksum: 0xb21b [correct]
    [Checksum Status: Good]
    [Stream index: 0]
    [Stream Packet Number: 1]
    [Timestamps]
    UDP payload (29 bytes)
Data (29 bytes)
    Data: 5dfa5ece7f3b6cc5fc2756aab695ff639f3089d8210c7890926cd88daf
    [Length: 29]
```

Manual calculation:
On wireshark right click UDP, copy->hex stream to get UDP header bytes:

c31301bb0025b21b

Given:

Source IP Address : 172.20.40.252

Destination IP Address : 142.251.150.119

Protocol : UDP (17)

UDP Length : 37 bytes

UDP Checksum (given) : 0xB21B

1. Construct Pseudo Header

Source IP (172.20.40.252)

AC14

28FC

Destination IP (142.251.150.119)

8EFB

9677

Protocol (UDP = 17)

0011

UDP Length (37 bytes)

0025

Pseudo Header:

AC14

28FC

8EFB

9677

0011

0025

2. UDP Header (Checksum set to 0000)

Original UDP Header:

C313

01BB

0025

B21B

Replace checksum with 0000:

C313

01BB

0025

0000

3. UDP Payload (29 bytes)

Payload (29 bytes):

5DFA5ECE7F3B6CC5FC2756AAB695FF639F3089D8210C7890926CD88DAF

Since payload length is odd (29 bytes), pad one byte 00 at the end.

Split into 16-bit words:

5DFA

5ECE

7F3B

6CC5

FC27

56AA

B695

FF63

9F30

89D8

210C

7890

926C

D88D

AF00

4. Perform 16-bit One’s Complement Addition

Add Pseudo Header:

AC14 + 28FC = D510

D510 + 8EFB = 1640B → wrap → 640C

640C + 9677 = FA83

FA83 + 0011 = FA94

FA94 + 0025 = FAB9

Add UDP Header:

FAB9 + C313 = 1BDCC → wrap → BDCD

BDCD + 01BB = BF88

BF88 + 0025 = BFAD

BFAD + 0000 = BFAD

Add Payload:

BFAD + 5DFA = 11DA7 → wrap → 1DA8

1DA8 + 5ECE = 7C76

7C76 + 7F3B = FBB1

FBB1 + 6CC5 = 16876 → wrap → 6877

6877 + FC27 = 1649E → wrap → 649F

649F + 56AA = BB49

BB49 + B695 = 171DE → wrap → 71DF

71DF + FF63 = 17142 → wrap → 7143

7143 + 9F30 = 11073 → wrap → 1074

1074 + 89D8 = 9A4C

9A4C + 210C = BB58

BB58 + 7890 = 133E8 → wrap → 33E9

33E9 + 926C = C655

C655 + D88D = 19EE2 → wrap → 9EE3

9EE3 + AF00 = 14DE3 → wrap → 4DE4

Final Sum = 4DE4

5. 1's Complement

FFFF - 4DE4 = B21B

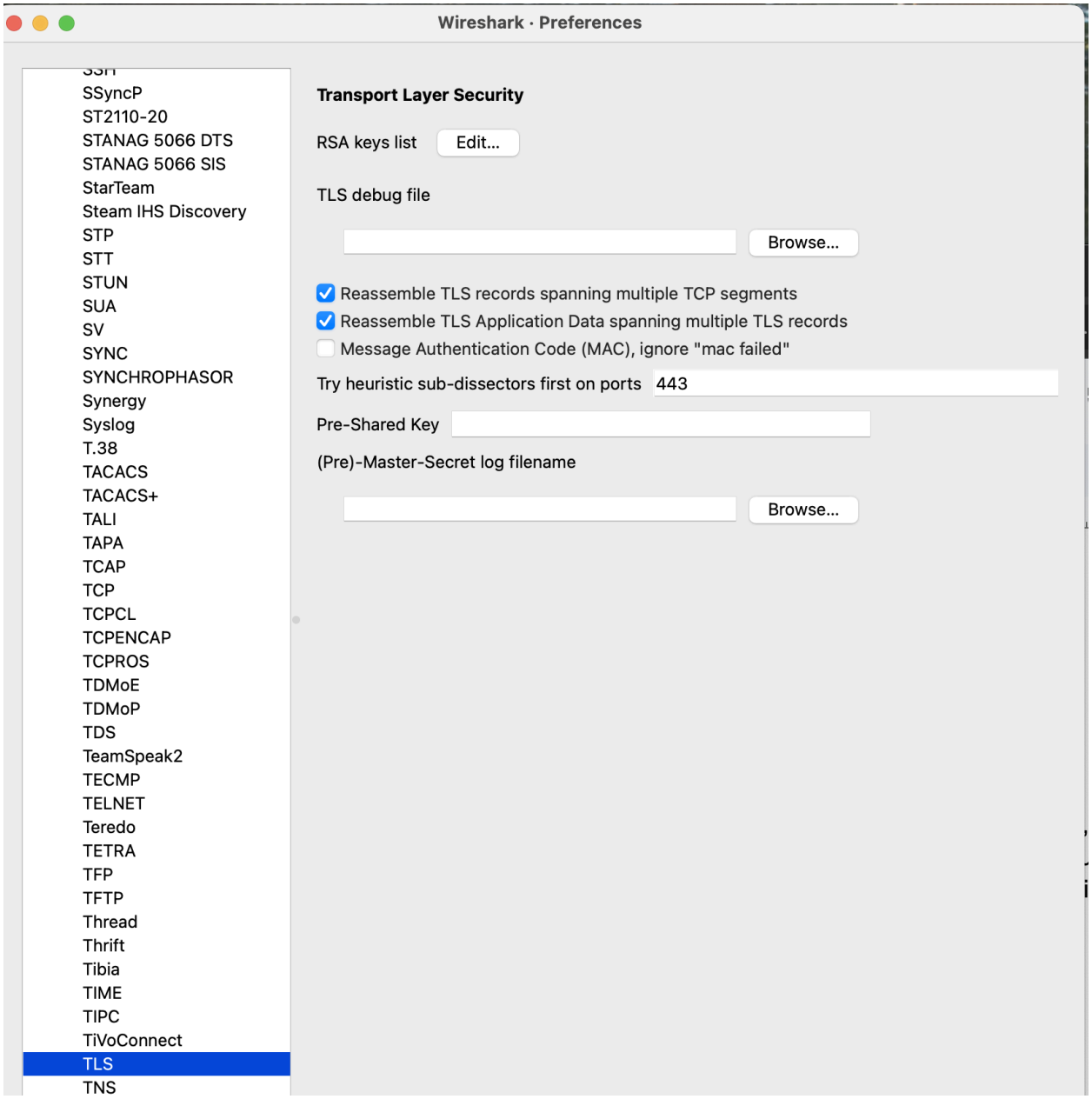
Checksum is valid

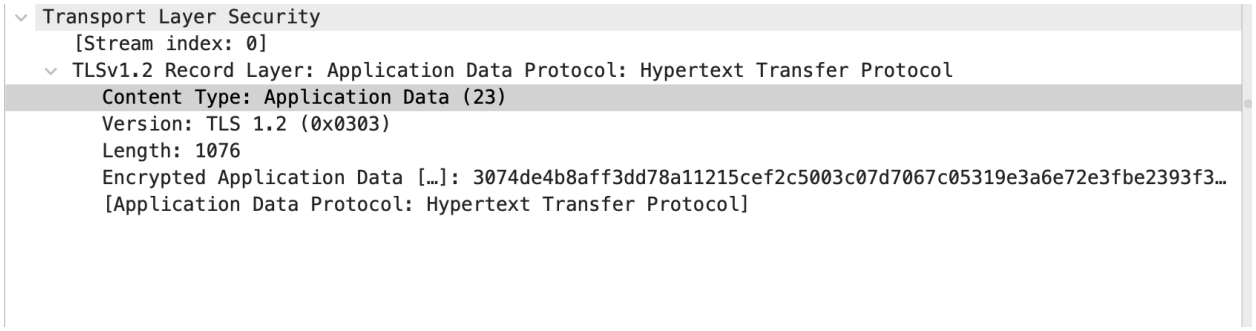
TLS:

Packet chosen:

tls

No.	Time	Source	Destination	Protocol	Info	Length	Source Port	Destination Port	Source HW Address	Destination HW Address
4	0.276369	172.20.40.252	182.79.245.149	TLSv1…	Application Data	1147	49946	443	82:39:5e:f8:7a:7b	Cisco_61:f5:1f





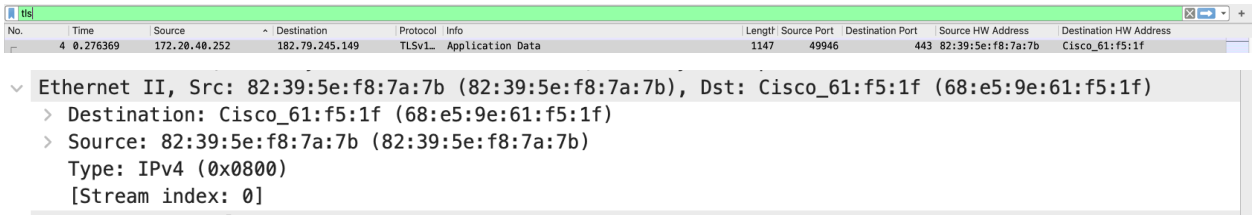
TLS does not contain a checksum field. Instead, it ensures integrity using

cryptographic mechanisms such as Message Authentication Codes (MAC) in TLS 1.2 or Authenticated Encryption (AEAD) in TLS 1.3.

Therefore, checksum validation is not applicable for TLS packets in Wireshark. Integrity verification occurs internally during cryptographic decryption.

ETHERNET:

Packet chosen:



Wireshark provides an option to validate Ethernet checksum (FCS) if present. However, in this capture file, the Ethernet Frame Check Sequence was not included because it is typically removed by the network interface card before reaching the operating system. Therefore, Ethernet checksum validation could not be performed in software.
Ethernet uses a 32-bit Frame Check Sequence (FCS) based on CRC-32 for error detection. The FCS is calculated and verified by the Network Interface Card (NIC) hardware.

In most packet captures, the Ethernet FCS field is not included because it is removed by hardware before the frame reaches the operating system.

Therefore, manual checksum validation for Ethernet is not performed in Wireshark or Scapy. Integrity verification is handled at the hardware level.

ICMP:

Packet chosen:

icmp

No.	Time	Source	Destination	Protocol	Info	Length	Source Port	Destination Port	Source HW Address	Destination HW Address
106	9.062501	172.20.40.252	142.251.222.206	ICMP	Echo (ping) request id=0x9a50, seq=0/0, ttl=64 …	98			82:39:5e:f8:7a:7b	Cisco_61:f5:1f

Wireshark checksum:

```
Internet Control Message Protocol
    Type: Echo (ping) request (8)
    Code: 0
    Checksum: 0x77bf [correct]
    [Checksum Status: Good]
    Identifier (BE): 39504 (0x9a50)
    Identifier (LE): 20634 (0x509a)
    Sequence Number (BE): 0 (0x0000)
    Sequence Number (LE): 0 (0x0000)
    [Response frame: 107]
    ICMP Data: 69886e89000122da08090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f202122232425262728292a2b2c2d2e2f3031323334353637
      Timestamp from icmp data: Feb  8, 2026 16:37:53.074458000 IST
      [Timestamp from icmp data (relative): 89.000 microseconds]
      Data (48 bytes)
        Data: 08090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f202122232425262728292a2b2c2d2e2f3031323334353637
        [Length: 48]
```

Manual Correction:
On wireshark right click ICMP, copy->hex stream to get ICMP header bytes:

080077bf9a50000069886e89000122da08090a0b0c0d0e0f1011121314151
61718191a1b1c1d1e1f202122232425262728292a2b2c2d2e2f3031323334
353637

1. Replace Checksum Field with 0000

Original first 8 bytes:

0800 77BF 9A50 0000

Replace checksum with 0000:

0800

0000

9A50

0000

2. Split Entire ICMP Packet into 16-bit Words

0800

0000

9A50

0000

6988

6E89

0001

22DA

0809

0A0B

0C0D

0E0F

1011

1213

1415

1617

1819

1A1B

1C1D

1E1F

2021

2223

2425

2627

2829

2A2B

2C2D

2E2F

3031

3233

3435

3637

(Total bytes are even, so no padding required.)

3. Perform 16-bit 1’s Complement Addition

0800 + 0000 = 0800

0800 + 9A50 = A250

A250 + 0000 = A250

A250 + 6988 = 10BD8 → wrap → 0BD9

0BD9 + 6E89 = 7A62

7A62 + 0001 = 7A63

7A63 + 22DA = 9D3D

9D3D + 0809 = A546

A546 + 0A0B = AF51

AF51 + 0C0D = BB5E

BB5E + 0E0F = C96D

C96D + 1011 = D97E

D97E + 1213 = EB91

EB91 + 1415 = 0FA6 → wrap

0FA6 + 1617 = 25BD

25BD + 1819 = 3DD6

3DD6 + 1A1B = 57F1

57F1 + 1C1D = 740E

740E + 1E1F = 922D

922D + 2021 = B24E

B24E + 2223 = D471

D471 + 2425 = F896

F896 + 2627 = 11EBD → wrap → 1EBE

1EBE + 2829 = 46E7

46E7 + 2A2B = 7112

7112 + 2C2D = 9D3F

9D3F + 2E2F = CB6E

CB6E + 3031 = FB9F

FB9F + 3233 = 12DD2 → wrap → 2DD3

2DD3 + 3435 = 6208

6208 + 3637 = 883F

Final Sum = 883F

4: 1’s Complement

FFFF - 883F = 77BF

Checksum is valid

SCAPY:

2 pcap files were used since the 1st pcap file did not contain ICMP packets.
Code:

```
from scapy.all import *
import sys
def validate_packet(pkt, index):
    print(f"\nPacket {index}")
    # Rebuild packet to force checksum recalculation
    rebuilt_pkt = pkt.__class__(bytes(pkt))
    # IP Checksum Validation
    if IP in pkt:
        original_ip = pkt[IP].chksum
        recalculated_ip = rebuilt_pkt[IP].chksum
        print("IP Checksum:")
        print(f"  Original     : {hex(original_ip)}")
        print(f"  Recalculated : {hex(recalculated_ip)}")
        print(f"  Status       : {'VALID' if original_ip == recalculated_ip else 'INVALID'}")
    # TCP Checksum Validation
    if TCP in pkt:
        original_tcp = pkt[TCP].chksum
        recalculated_tcp = rebuilt_pkt[TCP].chksum
        print("\nTCP Checksum:")
        print(f"  Original     : {hex(original_tcp)}")
        print(f"  Recalculated : {hex(recalculated_tcp)}")
        print(f"  Status       : {'VALID' if original_tcp == recalculated_tcp else 'INVALID'}")
    # UDP Checksum Validation
    if UDP in pkt:
        original_udp = pkt[UDP].chksum
        recalculated_udp = rebuilt_pkt[UDP].chksum
        print("\nUDP Checksum:")
        print(f"  Original     : {hex(original_udp)}")
        print(f"  Recalculated : {hex(recalculated_udp)}")
        print(f"  Status       : {'VALID' if original_udp == recalculated_udp else 'INVALID'}")
    # ICMP Checksum Validation
    if ICMP in pkt:
        original_icmp = pkt[ICMP].chksum
        recalculated_icmp = rebuilt_pkt[ICMP].chksum
        print("\nICMP Checksum:")
        print(f"  Original     : {hex(original_icmp)}")
        print(f"  Recalculated : {hex(recalculated_icmp)}")
        print(f"  Status       : {'VALID' if original_icmp == recalculated_icmp else 'INVALID'}")
# Main Execution
if len(sys.argv) != 2:
    print("Usage: python3 checksum_validation.py <pcap_file>")
    sys.exit(1)
pcap_file = sys.argv[1]
print(f"\nReading PCAP file: {pcap_file}")
packets = rdpcap(pcap_file)
print(f"Total Packets Found: {len(packets)}")
for i, packet in enumerate(packets, start=1):
    validate_packet(packet, i)
```

Sample Output:

```
maanyaagrawal@Maanyas-MacBook-Pro Desktop % python3 checksum_validation.py lab6capture.pcap

Reading PCAP file: lab6capture.pcap
Total Packets Found: 12903

Packet 1
IP Checksum:
  Original     : 0xd144
  Recalculated : 0xd144
  Status       : VALID

TCP Checksum:
  Original     : 0x7c53
  Recalculated : 0x7c53
  Status       : VALID

Packet 2
IP Checksum:
  Original     : 0xc140
  Recalculated : 0xc140
  Status       : VALID

TCP Checksum:
  Original     : 0x38a
  Recalculated : 0x38a
  Status       : VALID

Packet 3

Packet 4
IP Checksum:
  Original     : 0xb595
  Recalculated : 0xb595
  Status       : VALID

TCP Checksum:
  Original     : 0x5887
  Recalculated : 0x5887
  Status       : VALID

Packet 5
IP Checksum:
  Original     : 0xb595
  Recalculated : 0xb595
  Status       : VALID

TCP Checksum:
  Original     : 0x26ee
  Recalculated : 0x26ee
  Status       : VALID

Packet 6
IP Checksum:
  Original     : 0xb594
  Recalculated : 0xb594
  Status       : VALID

TCP Checksum:
  Original     : 0xaed0
  Recalculated : 0xaed0
  Status       : VALID

Packet 7
IP Checksum:
  Original     : 0xb592
  Recalculated : 0xb592
  Status       : VALID

TCP Checksum:
  Original     : 0x8e62
  Recalculated : 0x8e62
  Status       : VALID

Packet 8
```

```
maanyaagrawal@Maanyas-MacBook-Pro Desktop % python3 checksum_validation.py icmp_capture.pcap

Reading PCAP file: icmp_capture.pcap
Total Packets Found: 205

Packet 1
IP Checksum:
  Original     : 0xa046
  Recalculated : 0xa046
  Status       : VALID

UDP Checksum:
  Original     : 0x985e
  Recalculated : 0x985e
  Status       : VALID

Packet 2
IP Checksum:
  Original     : 0xa491
  Recalculated : 0xa491
  Status       : VALID

UDP Checksum:
  Original     : 0x5f5e
  Recalculated : 0x5f5e
  Status       : VALID

Packet 3
IP Checksum:
  Original     : 0xd631
  Recalculated : 0xd631
  Status       : VALID

TCP Checksum:
  Original     : 0x77ce
  Recalculated : 0x77ce
  Status       : VALID

Packet 4
IP Checksum:
  Original     : 0xf409
  Recalculated : 0xf409
  Status       : VALID

TCP Checksum:
  Original     : 0x282e
  Recalculated : 0x282e
  Status       : VALID

Packet 5
IP Checksum:
  Original     : 0xf3b3
  Recalculated : 0xf3b3
  Status       : VALID

TCP Checksum:
  Original     : 0x30e9
  Recalculated : 0x30e9
  Status       : VALID

Packet 6
IP Checksum:
  Original     : 0xd68c
  Recalculated : 0xd68c
  Status       : VALID

TCP Checksum:
  Original     : 0x204e
  Recalculated : 0x204e
  Status       : VALID
```